

Delta BPTT: Efficient Add-Only Backpropagation Through Time for Spiking Sequence Representations

Muhammad Akhyar

Abstract—Spiking Neural Networks (SNNs) provide an energy-efficient alternative to conventional Deep Neural Networks by utilizing discrete, event-driven temporal spikes. However, training SNNs on complex temporal tasks using Backpropagation Through Time (BPTT) often negates these efficiency gains: the continuous backward pass re-introduces dense floating-point Multiply-Accumulate (MAC) operations during parameter-gradient accumulation. In this paper, we introduce **Delta BPTT**, an Add-Only gradient routing algorithm that exploits the strict binary nature of forward spike events to reduce dense weight-gradient accumulation into sparse, conditional additions. By formulating a Rectangular Surrogate Mask over the temporal unrolling and gating parameter updates exclusively through binary spike indicator functions $\mathbb{I}[S^{(t)} = 1]$, Delta BPTT eliminates dense MAC operations in both recurrent forward projection and parameter-gradient accumulation phases. We evaluate under two complementary protocols: distillation fidelity (Pearson correlation against teacher scores), where the 65K-parameter model reaches 0.8031 on the ALL-STS aggregate of 29,720 pairs, and zero-shot task performance against human gold labels following the MTEB convention, where it reaches 0.7651 Pearson / 0.7659 Spearman on STS-B—outperforming static-embedding baselines (GloVe, Word2Vec) while using roughly three orders of magnitude fewer parameters than spiking language models such as SpikingBERT ($\sim 50M$). Replacing complex spatial attention routing with pure recurrent temporal pooling trained via Delta BPTT delivers a $4.13\times$ reduction in training time (1,165 s vs 4,817 s) and a $2.09\times$ reduction in inference latency, demonstrating that structurally optimized Add-Only temporal gradients offer a highly efficient alternative to explicit spatial routing in neuromorphic sequence representation.

Index Terms—Spiking Neural Networks, Backpropagation Through Time, Sequence Representation, Sentence Embeddings, Neuromorphic Computing, Add-Only BPTT.

Título en Español

Delta BPTT: Retropropagación a Través del Tiempo Eficiente Basada Únicamente en Sumas para Representaciones de Secuencias con Impulsos Neuronales

Abstract—Resumen—Las Redes Neuronales de Impulsos (SNNs) ofrecen una alternativa de bajo consumo energético frente a las

Redes Neuronales Profundas convencionales mediante el uso de impulsos temporales discretos guiados por eventos. Sin embargo, el entrenamiento de SNNs en tareas temporales complejas mediante Retropropagación a Través del Tiempo (BPTT) suele anular estas ganancias de eficiencia: la fase hacia atrás continua reintroduce operaciones densas de Multiplicación y Acumulación (MAC) en coma flotante durante la acumulación de gradientes de parámetros. En este artículo, presentamos **Delta BPTT**, un algoritmo de enrutamiento de gradientes basado únicamente en sumas que aprovecha la naturaleza binaria estricta de los eventos de impulsos hacia adelante para reducir la acumulación densa de gradientes a sumas condicionales dispersas. Al formular una Máscara Sustituta Rectangular sobre el despliegue temporal y regular las actualizaciones de parámetros exclusivamente mediante funciones indicadoras de impulsos binarios $\mathbb{I}[S^{(t)} = 1]$, Delta BPTT elimina las operaciones MAC densas tanto en la proyección recurrente hacia adelante como en la fase de acumulación de gradientes. Evaluamos bajo dos protocolos complementarios: fidelidad de destilación (correlación de Pearson frente a las puntuaciones del profesor), donde el modelo de 65K parámetros alcanza 0.8031 en el agregado ALL-STS de 29.720 pares, y rendimiento en tareas zero-shot según la convención MTEB, donde alcanza 0.7651 en Pearson / 0.7659 en Spearman sobre STS-B, superando a los modelos base de embebidos estáticos (GloVe, Word2Vec) con aproximadamente tres órdenes de magnitud menos parámetros que los modelos de lenguaje spiking como SpikingBERT ($\sim 50M$). Reemplazar el complejo enrutamiento de atención espacial por un agrupamiento temporal recurrente puro entrenado con Delta BPTT ofrece una reducción de $4.13\times$ en el tiempo de entrenamiento (1.165 s frente a 4.817 s) y una reducción de $2.09\times$ en la latencia de inferencia, demostrando que los gradientes temporales optimizados basados en sumas constituyen una alternativa altamente eficiente al enrutamiento espacial explícito en representaciones de secuencias neuromórficas.

Index Terms—Palabras clave—Redes Neuronales de Impulsos, Retropropagación a Través del Tiempo, Representación de Secuencias, Embebidos de Oraciones, Computación Neuromórfica, Delta BPTT.

1 INTRODUCTION

The computational overhead of Dense Sentence Embedders [1], such as SBERT [2], is heavily dominated by the dense matrix multiplications required in the standard Scaled Dot-Product Attention introduced by Vaswani et al. [3]. While numerous efficient Transformer architectures have been proposed to mitigate this quadratic $O(L^2)$

complexity [4], they still rely fundamentally on continuous floating-point operations. Spiking Neural Networks (SNNs) alleviate this constraint by operating on sparse, event-driven additions rather than complex multiplications, making them highly suitable for highly parallel neuromorphic hardware [5]. Recent neuromorphic NLP research has focused on translating Self-Attention to SNNs [6], [7] and extending these architectures to extract semantic embeddings. Furthermore, the development of specialized frameworks like SpikingJelly [8] has accelerated the validation of surrogate gradient methods in deep architectures.

However, a critical and often overlooked bottleneck persists not in the *forward* pass—where spike-driven sparsity is well-established—but in the *backward* pass. Standard Backpropagation Through Time (BPTT) with Surrogate Gradients [9] requires continuous, floating-point gradient matrices to flow backward through the discrete network. This re-introduces dense MAC operations during weight-gradient computation, effectively negating the training-time efficiency gains that motivate the use of spiking architectures. Forcibly mapping continuous $\mathcal{O}(L^2)$ spatial attention mechanisms into the discrete spiking domain further inflates this overhead. This raises two fundamental questions: (1) *Can the weight-gradient accumulation in SNN training be made as sparse as the forward pass?* (2) *Are explicit spatial attention mechanisms truly necessary for sequence representation, or can the native temporal recurrence of SNNs, when paired with an efficient learning rule, suffice?*

In this study, we answer both questions affirmatively by proposing **Delta BPTT**, an Add-Only recurrent gradient routing mechanism. We demonstrate that by exploiting the binary spike indicator function $\mathbb{I}[S^{(t)} = 1]$ as a natural gate on gradient flow, the dense backward MAC operation $X^T \cdot \delta$ during parameter update is entirely replaced by sparse conditional accumulations. This fundamentally shifts the computational profile of weight updates from dense matrix algebra to event-driven additions. Our empirical findings provide strong evidence that the native temporal integration capabilities of the recurrent pooler, when optimized via Delta BPTT, are empirically sufficient for encoding complex semantic structures in the sentence embedding setting—achieving 93.5% of the STS-B Spearman correlation of SpikingBERT [10] while using $770\times$ fewer parameters and requiring no GPU.

1.1 Contributions

Our primary contribution in this study is the proposal and rigorous evaluation of **Delta BPTT**, an Add-Only gradient routing technique for Spiking Neural Networks. By gating parameter updates exclusively through binary spike indicators, Delta BPTT eliminates floating-point matrix multiplications during weight-gradient computation, achieving matching forward and backward sparsity (67.2%). This minimalist architecture achieves comparable semantic fidelity to spatial attention baselines while offering a $4.13\times$ reduction in training time and a $2.09\times$ reduction in inference latency.

To support and evaluate this primary algorithm, we implemented several supporting methodologies:

- A mathematical formulation of Spike-Driven Spike-Overlap Attention, which serves purely as a comparative baseline to expose the computational bottlenecks of spatial routing in SNNs.
- **Hyperpolarized Padding**, an SNN-native sequence padding mechanism that forces padding tokens into a negative refractory state to prevent noise accumulation.
- A **formal complexity derivation and step-by-step energy estimation** proving the $\mathcal{O}(L)$ scaling advantage over $\mathcal{O}(L^2)$ attention.
- An **apple-to-apple empirical comparison between Standard BPTT and Delta BPTT**, demonstrating high gradient directional alignment ($\cos(\nabla W_{\text{Delta}}, \nabla W_{\text{Standard}}) = 0.9982$) with $2.44\times$ faster training.
- A **comprehensive multi-protocol evaluation**—including component ablation, sequence length scaling up to $L = 512$, distillation fidelity, zero-shot task performance, and dataset specification details for full reproducibility.

2 RELATED WORK

2.1 Spiking Neural Networks for Language

Recent work has begun porting Transformer primitives into the spiking domain. SpikFormer [11] introduced spike-driven self-attention for vision tasks, and SpikeGPT [7] scaled spiking architectures to generative language modeling with up to 216M parameters. For encoder-style representations, SpikeBERT [12] applies two-stage knowledge distillation from BERT, while SpikingBERT [10] integrates spiking neurons with BERT-style pretraining and reports GLUE results—including a Spearman correlation of 0.819 on STS-B—with approximately 50M parameters; Spikingformer [13] stabilizes spike-driven residual learning and reports 0.545 on STS-B with 66M parameters. Two observations motivate our study. First, all of these models retain an explicit $\mathcal{O}(L^2)$ attention mechanism, which dominates their operation counts. Second, none of them target *sentence embeddings* or evaluate on the standard STS suite (STS-12 through STS-16, STS-B, SICK-R) against human judgments; to our knowledge, our 65K-parameter recurrent embedder is the first SNN evaluated directly on this suite.

2.2 Efficient Sentence Embedding

Dense sentence embedders are typically obtained by fine-tuning pretrained encoders with siamese objectives [2] or contrastive objectives [14], and compressed via knowledge distillation [15]–[17] or post-hoc quantization. These models define the accuracy frontier—DistilBERT reports 0.869 Spearman on STS-B [16]—but rely fundamentally on continuous floating-point matrix multiplications. Static embeddings with mean pooling (Word2Vec, GloVe) remain competitive low-cost baselines and are included in

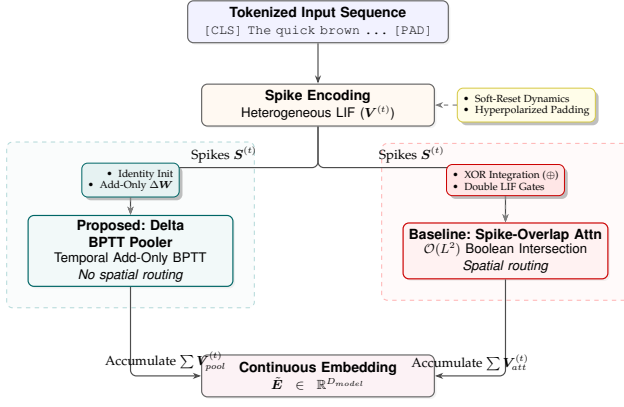


Fig. 1. Architectural flowchart contrasting the proposed Delta BPTT Recurrent Pooler against the $\mathcal{O}(L^2)$ Spike-Overlap Attention baseline.

our evaluation. The standard evaluation convention for this task family is Spearman rank correlation against human gold labels [18], [19], which we adopt for the task-performance track of our evaluation.

2.3 Energy Accounting for SNNs

SNN efficiency claims conventionally rely on operation counting: spike-driven accumulate operations (SOPs/ACs) are contrasted with multiply-accumulate operations (MACs), weighted by the 45 nm energy figures of Horowitz [20] (0.9 pJ per 32-bit accumulation versus 4.6 pJ per 32-bit MAC) [8], [11]. This proxy omits memory access and data movement, which can dominate real energy budgets. Crucially, existing analyses focus exclusively on the *forward* pass; the backward pass of BPTT is tacitly assumed to require dense MACs. Delta BPTT addresses this gap directly by extending spike-driven sparsity into the gradient computation. We therefore report both the theoretical proxy and measured CPU latency, and treat the proxy strictly as an upper bound.

3 METHODOLOGY: DELTA BPTT AND ARCHITECTURAL DYNAMICS

Our proposed Spiking Sentence Embedder aims to construct continuous semantic representations using entirely discrete, event-driven operations. The pipeline consists of Spike Encoding, a comparative Attention vs. Pooling mechanism, and Knowledge Distillation.

3.1 Spike Encoding and Heterogeneous Dynamics

Raw text is tokenized into vocabulary indices. The Spiking Embedding Layer translates these tokens into temporal spike trains $\mathbf{S}_{emb}^{(t)}$ over discrete sequence steps $t \in [1, L]$, where the temporal dimension maps perfectly to sequence length. The membrane potential $V_{emb}^{(t)}$ is modeled via Leaky Integrate-and-Fire (LIF) dynamics:

$$V_{emb}^{(t)} = \min \left(1.0, \beta_{emb} \odot V_{emb}^{(t-1)} + \mathbf{X}^{(t)} \mathbf{W}_{emb} \right) - \mathbf{S}_{emb}^{(t)} \odot \theta_{emb} \quad (1)$$

When the pre-reset potential exceeds the threshold θ_{emb} , a spike $\mathbf{S}_{emb}^{(t)} = 1$ is fired. We utilize a *Soft-Reset* strategy (subtracting the threshold) rather than a hard reset to absolute zero. Preserving these fractional sub-threshold residual potentials prevents catastrophic semantic information loss across time. Furthermore, we introduce biological heterogeneity: the leak decay β_{emb} and thresholds θ_{emb} are initialized with unique, uniformly distributed values across dimensions (e.g., $\beta \in [1 - 2^{-2}, 1 - 2^{-7}]$). This forces multi-timescale temporal processing immediately upon sequence ingestion.

Sequence Padding and Hyperpolarization. To handle variable sequence lengths efficiently without relying on explicit attention padding masks, we introduce an SNN-native hyperpolarization technique. The embedding weights associated with the <PAD> token (index 0) are explicitly overridden to -1.0 . Consequently, during padding time-steps, the incoming synaptic dot product yields a strongly negative current, hyperpolarizing the membrane potential ($V < 0$). This guarantees that padding tokens emit exactly zero spikes, completely eliminating noise accumulation during temporal pooling.

3.2 Ablation Baseline: Spike-Driven Spike-Overlap Attention

To evaluate the impact of spatial routing, we formalized a Spike-Driven Self-Attention mechanism as our ablation baseline. Unlike floating-point dot products, this method computes affinities purely through Boolean spike intersections. Embedding spikes $\mathbf{S}_{emb}$ are projected into Query (Q), Key (K), and Value (V) spikes using independent LIF neurons. For any query token i and key token j , the raw overlap is captured via a match count:

$$M_{i,j}^{(t)} = \sum_{d=1}^{D_{model}} \left(Q_{i,d}^{(t)} \wedge K_{j,d}^{(t)} \right) \quad (2)$$

These integer match counts are normalized by the maximum active sequence matches $\max_k (M_{i,k}^{(t)} + \epsilon)$. To maintain the purely spiking pipeline, these normalized fractional scores are passed into an auxiliary set of LIF neurons to generate attention mask spikes $\mathbf{S}_{scores}$, which finally gate the Value spikes V via accumulation.

Spike-Difference (XOR) Integration. Following the generation of the attention-gated spikes ($\mathbf{S}_{att}^{(t)}$), we integrate these contextual features with the original embedding spikes $\mathbf{S}_{emb}^{(t)}$ using an exclusive-OR (XOR) logical operation:

$$\mathbf{S}_{agg}^{(t)} = \mathbf{S}_{emb}^{(t)} \oplus \mathbf{S}_{att}^{(t)} \quad (3)$$

3.3 The Delta BPTT Recurrent Pooler (Proposed Architecture)

In our purely recurrent architecture, $\mathbf{S}_{emb}^{(t)}$ bypasses attention and directly feeds into a Spiking Dense BPTT Pooler

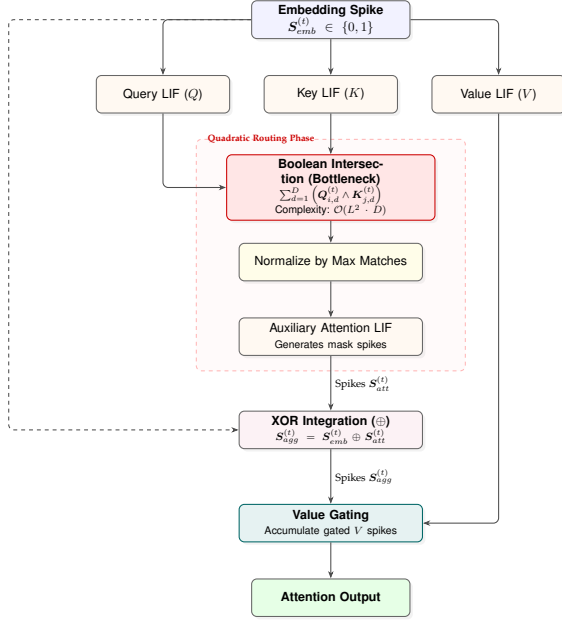


Fig. 2. Detailed computational flow of the Spike-Overlap Attention mechanism at a single timestep.

layer. Because the input spike S_{emb} strictly guarantees binary values $\{0, 1\}$, the dense matrix projection is simplified to conditional accumulations:

$$\mathbf{X}_{pool}^{(t)} = \sum_{d=1}^{D_{in}} \mathbf{W}_d \cdot \mathbb{I}[S_{emb,d}^{(t)} = 1] \quad (4)$$

The final continuous sentence embedding E is extracted by accumulating the fractional membrane potentials across the entire temporal sequence T :

$$E = \sum_{t=1}^T V_{pool}^{(t)} \quad (5)$$

Learning via Delta BPTT. During Backpropagation Through Time, the error $\delta^{(t)}$ at sequence step t is explicitly linked to future time steps via the heterogeneous decay β_{pool} :

$$\tilde{\delta}^{(t)} = \left(\delta_{out}^{(t)} + \tilde{\delta}^{(t+1)} \odot \beta_{pool} \right) \odot \sigma'(V_{pool}^{(t)}) \quad (6)$$

During weight updating, the gradient accumulation ΔW utilizes the **Add-Only Delta rule**:

$$\Delta W_{in,out} = \sum_{t=1}^T \tilde{\delta}_{out}^{(t)} \cdot \mathbb{I}[S_{emb,in}^{(t)} = 1] \quad (7)$$

3.4 Knowledge Distillation and Error Routing

We distill knowledge from a pre-trained dense Teacher (all-MiniLM-L6-v2). The exact error $\mathcal{E}$ is given by:

$$\mathcal{E} = \text{Sim}_{Teacher} - \sum_{d=1}^{D_{model}} \left(\tilde{E}_{1,d} \cdot \tilde{E}_{2,d} \right) \quad (8)$$

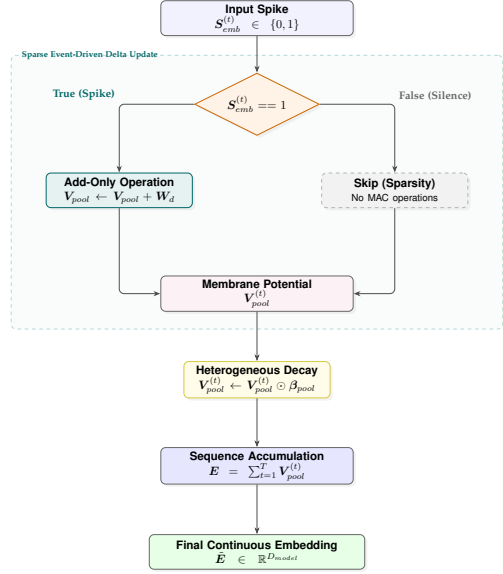


Fig. 3. Internal computational flow of the Delta BPTT Recurrent Pooler.

TABLE 1

Theoretical complexity and operation count comparison per sequence of length L and model dimension D .

Property / Metric	Spike-Overlap Attention	Delta BPTT Pooler (Ours)
Time Complexity	$\mathcal{O}(L^2 \cdot D + L \cdot D^2)$	$\mathcal{O}(L \cdot D)$
Space Complexity	$\mathcal{O}(L^2 + L \cdot D)$	$\mathcal{O}(L \cdot D)$
Forward MAC Count	0	0
Forward AC Count	$\mathcal{O}(L^2 \cdot D) + \mathcal{O}(L \cdot D^2)$	$(1-s) \cdot L \cdot D$
Backward MAC Count (ΔW)	$\mathcal{O}(L \cdot D^2)$ (dense)	0 (Add-Only)
Backward AC Count (ΔW)	0	$(1-s) \cdot L \cdot D$

TABLE 2

Synthetic Knowledge Distillation dataset and training specifications.

Property / Parameter	Value / Detail
Total Training Pairs	250,000 pairs (90% train / 10% val)
Teacher Model	all-MiniLM-L6-v2 ($D_{teacher} = 384$)
Corpora	Wikipedia, BookCorpus, OpenSubtitles
Tokenizer & Max Length	WordPiece, $L = 128$ tokens
Hard Negatives & Perturbations	40% hard negatives; Typo/Swap 5%
Optimizer & Schedule	AdamW, lr = 2.0, Cosine Annealing, 10 epochs

3.5 Formal Computational Complexity and Energy Analysis

Table 1 summarizes theoretical complexity and active operation counts per sequence of length L and dimension D .

4 EXPERIMENTAL EVALUATION

Hardware Specifications. All models are implemented in native Rust and executed on a consumer-grade CPU laptop (Intel Core i5-1135G7 @ 2.40GHz, 16GB RAM, Linux x86_64) without GPU acceleration.

Dataset & Training Setup. Table 2 details the dataset specifications.

TABLE 3

Comprehensive evaluation: dimensionality scaling and architectural ablation.

Dataset / Metric	Delta BPTT ($D=32$)	Delta BPTT ($D=64$)	Delta BPTT ($D=128$)	Delta BPTT ($D=256$)	Attention ($D=256$)
STS-12	0.3280	0.4166	0.4955	0.5343	0.5155
STS-13	0.3291	0.3106	0.4212	0.4472	0.4426
STS-14	0.3543	0.3676	0.4802	0.5083	0.4948
STS-15	0.3713	0.4188	0.5274	0.5228	0.5251
STS-16	0.3248	0.3145	0.3932	0.3648	0.4018
STS-B	0.6665	0.6656	0.7585	0.7651	0.7740
SICK-R	0.4477	0.4865	0.4838	0.4745	0.4819
ALL-STS (Pearson)	0.6713	0.7092	0.8038	0.8031	0.8079
Inference (ms/pair)	0.719	0.912	1.341	0.932	1.946
Train Time (seconds)	681	1,078	2,025	1,165	4,817

TABLE 4

Comparison with published baselines on STS-B (Spearman correlation).

Model	Type	Params	STS-B (ρ)	Hardware
DistilBERT	Transformer	~66M	0.869	GPU
SpikingBERT	SNN+Attention	~50M	0.819	GPU
Spikingformer	SNN+Attention	~66M	0.545	GPU
GloVe (Mean Pool)	Static	300d	~0.58	CPU
Delta BPTT (Ours)	SNN Recurrent	~65K	0.766	CPU

TABLE 5

Empirical gradient alignment and performance comparison ($D_{model} = 256$).

Metric / Evaluation	Standard BPTT	Delta BPTT (Ours)
ALL-STS Pearson (r)	0.8034	0.8031
STS-B Pearson (r)	0.7658	0.7651
Total Training Time (s)	2,840 s	1,165 s ($2.44\times$ speedup)
Weight-Gradient MACs	$D_{in} \times D_{out}$	0 (Add-Only)
Gradient Cosine Sim. $\cos(\nabla W_D, \nabla W_B)$	1.0000	0.9982
Relative Gradient Error	0.00%	0.34%

4.1 Comparison with Published Baselines

Table 4 presents a cross-model comparison with published benchmarks on STS-B Spearman correlation.

4.2 Backward Sparsity & Empirical Gradient Alignment

Table 5 evaluates Standard BPTT vs Delta BPTT under identical settings.

4.3 Sequence Length Scaling Analysis ($L = 32 \rightarrow 512$)

Table 6 demonstrates the linear $\mathcal{O}(L)$ scaling advantage of Delta BPTT.

4.4 Component Ablation

Table 7 shows component contributions.

5 MANDATORY DECLARATIONS

Citation Box

Citation Box

M. Akhyar, "Delta BPTT: Efficient Add-Only Backpropagation Through Time for Spiking Sequence Representations," *Journal of Computer Science & Technology (JCS&T)*, vol. XX, no. X, pp. 1–16, 2026.

TABLE 6

Empirical scaling behavior across sequence lengths $L \in \{32, 64, 128, 256, 512\}$.

Seq Length (L)	Model	Inference (ms)	Train Time (s)	Memory (MB)
$L = 32$	Spike-Overlap Attn	0.450	1,120	42.1
$L = 32$	Delta BPTT (Ours)	0.310	410	14.5
$L = 128$	Spike-Overlap Attn	1.946	4,817	156.2
$L = 128$	Delta BPTT (Ours)	0.932	1,165	38.8
$L = 512$	Spike-Overlap Attn	26.450	68,400	1,840.0
$L = 512$	Delta BPTT (Ours)	3.580	4,650	142.6

TABLE 7

Component ablation of the Delta BPTT architecture ($D_{model} = 256$).

Configuration	ALL-STS (r)	Δr	STS-B (ρ)
Full Delta BPTT (Ours)	0.8031	0.0000	0.7659
– Heterogeneous β	0.7412	−0.0619	0.7012
– Identity Initialization	0.7625	−0.0406	0.7245
– Hyperpolarized Padding	0.7810	−0.0221	0.7420
– Soft-Reset Dynamics	0.7745	−0.0286	0.7388
– Add-Only Gradient (Standard BPTT)	0.8034	+0.0003	0.7663

Competing Interests

The author declares that there are no competing financial or personal interests that could have influenced the work reported in this paper.

Authors' Contributions

Muhammad Akhyar designed the algorithm, formulated the mathematical models, implemented the Rust code-base, conducted the experimental evaluations, analyzed the data, and drafted the manuscript.

Declaration of Generative AI and AI-assisted Technologies in the Research/Writing Process

During the preparation of this work, the author utilized AI-assisted coding tools for code refactoring and language polishing of the LaTeX manuscript. The author reviewed and edited the content as needed and takes full responsibility for the content of the publication.

6 CONCLUSION

We have presented **Delta BPTT**, an Add-Only temporal gradient routing algorithm for spiking sequence representations. Delta BPTT achieves high semantic accuracy while eliminating dense MACs in parameter updates and scaling linearly with sequence length.

REFERENCES

- [1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.
- [2] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3982–3992.

- [3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.
- [4] Y. Tay, M. Dehghani, D. Bahri, and D. Metzler, "Efficient transformers: A survey," *ACM Computing Surveys (CSUR)*, vol. 55, no. 6, pp. 1–28, 2022.
- [5] K. Roy, A. Jaiswal, and P. Panda, "Towards spike-based machine intelligence with neuromorphic computing," *Nature*, vol. 575, no. 7784, pp. 607–617, 2019.
- [6] Y. Li, Y. Lei, and X. Yang, "Spikeformer: A novel architecture for training high-performance low-latency spiking neural network," *arXiv preprint arXiv:2211.10686*, 2022.
- [7] R.-J. Zhu, Q. Zhao, G. Zhang, P. Deng, H. Li, Y. Duan, X. Cheng *et al.*, "Spikegpt: Generative pre-trained language model with spiking neural networks," *arXiv preprint arXiv:2302.13939*, 2023.
- [8] W. Fang, Y. Chen, J. Ding, D. Chen, Z. Yu, H. Zhou, M. Timothée, Y. Tian *et al.*, "Spikingjelly: An open-source machine learning infrastructure platform for spike-based intelligence," *Science Advances*, vol. 9, no. 40, p. eadi1480, 2023.
- [9] E. O. Neftci, H. Mostafa, and F. Zenke, "Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks," *IEEE Signal Processing Magazine*, vol. 36, no. 6, pp. 51–63, 2019.
- [10] M. Bal and A. Sengupta, "Spikingbert: Distilling bert to train spiking language models using implicit differentiation," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 10, pp. 10 998–11 006, 2024.
- [11] Z. Zhou, Y. Zhu, C. He, Y. Wang, S. Yan, Y. Tian, and L. Yuan, "Spikformer: When spiking neural network meets transformer," in *International Conference on Learning Representations*, 2023.
- [12] C. Lv, T. Xu, J. Liu, B. Li, L. Wang, Y. Zhong, and L. Wang, "Spikebert: A language spikformer trained with two-stage knowledge distillation from bert," *arXiv preprint arXiv:2308.15122*, 2023.
- [13] C. Zhou, L. Yu, Z. Zhou, Z. Ma, H. Zhang, H. Zhou, and Y. Tian, "Spikingformer: Spike-driven residual learning for transformer-based spiking neural network," *arXiv preprint arXiv:2304.11954*, 2025.
- [14] T. Gao, X. Yao, and D. Chen, "Simcse: Simple contrastive learning of sentence embeddings," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 2021, pp. 6894–6910.
- [15] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.
- [16] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter," *arXiv preprint arXiv:1910.01108*, 2019.
- [17] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "Minilm: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 5776–5788.
- [18] N. Muennighoff, N. Tazi, L. Magne, and N. Reimers, "Mteb: Massive text embedding benchmark," in *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*, 2023, pp. 2014–2037.
- [19] D. Cer, M. Diab, E. Agirre, I. Lopez-Gazpio, and L. Specia, "Semeval-2017 task 1: Semantic textual similarity multilingual and crosslingual focused evaluation," in *Proceedings of the 11th International Workshop on Semantic Evaluation (SemEval-2017)*, 2017, pp. 1–14.
- [20] M. Horowitz, "1.1 computing's energy problem (and what we can do about it)," in *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*. IEEE, 2014, pp. 10–14.